

```
1 !pip install -q ultralytics kaggle opencv-python matplotlib
```

1.2/1.2 MB 33.5 MB/s eta 0:00:00

```
1 !cd /content/major_project
```

```
1 !mkdir -p ~/.kaggle
2 !cp /content/kaggle.json ~/.kaggle/
3 !chmod 600 ~/.kaggle/kaggle.json
4
```

```
1 !kaggle datasets download \
2 -d rawsi18/military-assets-dataset-12-classes-yolo8-format \
3 -p /content/major_project \
4 --unzip
5
```

Dataset URL: <https://www.kaggle.com/datasets/rawsi18/military-assets-dataset-12-classes-yolo8-format>
License(s): Attribution 4.0 International (CC BY 4.0)
Downloading military-assets-dataset-12-classes-yolo8-format.zip to /content/major_project
99% 3.81G/3.83G [00:44<00:00, 86.5MB/s]
100% 3.83G/3.83G [00:44<00:00, 92.2MB/s]

```
1 yml_text = ""
2 path: /content/major_project/military_object_dataset
3 train: train/images
4 val: val/images
5 test: test/images
6
7 names:
8 0: camouflage_soldier
9 1: weapon
10 2: military_tank
11 3: military_truck
12 4: military_vehicle
13 5: civilian
14 6: soldier
15 7: civilian_vehicle
16 8: military_artillery
17 9: trench
18 10: military_aircraft
19 11: military_warship
```

```
20 ""
21
22 with open(
23     "/content/major_project/military_object_dataset/military_dataset.yaml",
24     "w"
25 ) as f:
26     f.write(yaml_text)
27
28 print("✅ YAML fixed and saved correctly")
29
```

✅ YAML fixed and saved correctly

```
1 import torch
2 print(torch.cuda.is_available())
3 print(torch.cuda.device_count())
4
```

True
1

```
1 from ultralytics import YOLO
2
3 model = YOLO("yolo11n.pt")
4
5 model.train(
6     data="/content/major_project/military_object_dataset/military_dataset.yaml",
7     epochs=5,
8     imgsz=416,
9     batch=4,
10    device=0,
11    project="/content/major_project",
12    name="runs"
13 )
14
```

```

0.79279, 0.79379, 0.79479, 0.7958, 0.7968, 0.7978, 0.7988, 0.7998, 0.8008, 0.8018,
0.8028, 0.8038, 0.8048, 0.80581, 0.80681, 0.80781, 0.80881, 0.80981, 0.81081, 0.81181, 0.81281,
0.81381, 0.81481, 0.81582,
0.81682, 0.81782, 0.81882, 0.81982, 0.82082, 0.82182, 0.82282, 0.82382, 0.82482, 0.82583,
0.82683, 0.82783, 0.82883, 0.82983, 0.83083, 0.83183, 0.83283, 0.83383, 0.83483, 0.83584, 0.83684,
0.83784, 0.83884, 0.83984,
0.84084, 0.84184, 0.84284, 0.84384, 0.84484, 0.84585, 0.84685, 0.84785, 0.84885, 0.84985,
0.85085, 0.85185, 0.85285, 0.85385, 0.85485, 0.85586, 0.85686, 0.85786, 0.85886, 0.85986, 0.86086,
0.86186, 0.86286, 0.86386,
0.86486, 0.86587, 0.86687, 0.86787, 0.86887, 0.86987, 0.87087, 0.87187, 0.87287, 0.87387,
0.87487, 0.87588, 0.87688, 0.87788, 0.87888, 0.87988, 0.88088, 0.88188, 0.88288, 0.88388, 0.88488,
0.88589, 0.88689, 0.88789,
0.88889, 0.88989, 0.89089, 0.89189, 0.89289, 0.89389, 0.89489, 0.8959, 0.8969, 0.8979,
0.8989, 0.8999, 0.9009, 0.9019, 0.9029, 0.9039, 0.9049, 0.90591, 0.90691, 0.90791, 0.90891,
0.90991, 0.91091, 0.91191,
0.91291, 0.91391, 0.91491, 0.91592, 0.91692, 0.91792, 0.91892, 0.91992, 0.92092, 0.92192,
0.92292, 0.92392, 0.92492, 0.92593, 0.92693, 0.92793, 0.92893, 0.92993, 0.93093, 0.93193, 0.93293,
0.93393, 0.93493, 0.93594,
0.93694, 0.93794, 0.93894, 0.93994, 0.94094, 0.94194, 0.94294, 0.94394, 0.94494, 0.94595,
0.94695, 0.94795, 0.94895, 0.94995, 0.95095, 0.95195, 0.95295, 0.95395, 0.95495, 0.95596, 0.95696,
0.95796, 0.95896, 0.95996,
0.96096, 0.96196, 0.96296, 0.96396, 0.96496, 0.96597, 0.96697, 0.96797, 0.96897, 0.96997,
0.97097, 0.97197, 0.97297, 0.97397, 0.97497, 0.97598, 0.97698, 0.97798, 0.97898, 0.97998, 0.98098,
0.98198, 0.98298, 0.98398,
0.98498, 0.98599, 0.98699, 0.98799, 0.98899, 0.98999, 0.99099, 0.99199, 0.99299, 0.99399,
0.99499, 0.996, 0.997, 0.998, 0.999, 1]), array([[0.97255, 0.97255, 0.96863, ..., 0,
0,
0],
[ 0.71229, 0.71229, 0.68715, ..., 0, 0, 0],
[ 0.95411, 0.95411, 0.94908, ..., 0, 0, 0],
...,
[ 0.91453, 0.91453, 0.89744, ..., 0, 0, 0],
[ 0, 0, 0, ..., 0, 0, 0],
[ 0.9191, 0.9191, 0.91439, ..., 0, 0, 0]]), 'Confidence', 'Recall'])
fitness: np.float64(0.24390044156188653)
keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metrics/mAP50-95(B)']
maps: array([ 0.35104, 0.33007, 0.49316, 0.20214, 0.19399, 0, 0.34599, 0.072174, 0.12728, 0,
0.56706, 0.2439])
names: {0: 'camouflage_soldier', 1: 'weapon', 2: 'military_tank', 3: 'military_truck', 4: 'military_vehicle', 5: 'civilian', 6: 'soldier', 7:
'civilian_vehicle', 8: 'military_artillery', 9: 'trench', 10: 'military_aircraft', 11: 'military_warship'}
nt_per_class: array([ 510, 358, 1787, 148, 307, 1, 745, 42, 117, 3, 1063, 0])
nt_per_image: array([385, 222, 938, 84, 149, 1, 420, 18, 85, 1, 594, 0])
results_dict: {'metrics/precision(B)': 0.47887404768872793, 'metrics/recall(B)': 0.42457035075544725, 'metrics/mAP50(B)': 0.3959736522402891,
'metrics/mAP50-95(B)': 0.24390044156188653, 'fitness': 0.24390044156188653}
save_dir: PosixPath('/content/major_project/runs')
speed: {'preprocess': 0.13126018327132102, 'inference': 3.0416414250267376, 'loss': 0.0028295137695271777, 'postprocess': 1.871114519891845}
stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [], 'target_img': []}
task: 'detect'

```

```
1 from google.colab import files
2 uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving ecb172dfca44fa000b6ae85f7f1cfccb.jpg to ecb172dfca44fa000b6ae85f7f1cfccb.jpg

```
1 from ultralytics import YOLO
2 import matplotlib.pyplot as plt
3 import cv2
4 import os
5 # Load trained model
6
7 model = YOLO("/content/major_project/runs/weights/best.pt")
8
9 # Get uploaded image name
10 img_name = list(uploaded.keys())[0]
11 # Predict
12 results = model.predict(source=img_name, save=True, conf=0.25)
13 # Output saved path
14 out_path = results[0].save_dir + "/" + os.path.basename(img_name)
15 # Show output image
16 img = cv2.imread(out_path)
17 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
18 plt.figure(figsize=(10,8))
19 plt.imshow(img)
20 plt.axis("off")
21 plt.show()
22 print("✅ Output saved at:", out_path)
```

image 1/1 /content/ecb172dfca44fa000b6ae85f7f1cfccb.jpg: 288x416 1 camouflage_soldier, 51.6ms
Speed: 1.9ms preprocess, 51.6ms inference, 5.1ms postprocess per image at shape (1, 3, 288, 416)
Results saved to /content/runs/detect/predict



✓ Output saved at: /content/runs/detect/predict/ecb172dfca44fa000b6ae85f7f1cfccb.jpg